

Financial Econometrics - PS2

Francesco Zucca, Michael Fehl, Giuseppe Lista

2026-02-12

Introduction

In this practice, we will work on identifying ARIMA(p, d, q)(P, D, Q)s models for non-stationary time series that have a seasonal component. We will also focus on validating models fitted to time series data and selecting the best model among different alternatives. The chosen model will then be used to generate forecasts. As done in previous practices, each student group must follow the practice script below, performing the necessary calculations and graphs in the R environment. The obtained results and answers to the questions should be compiled into a document containing your solution, created, for example, in R Markdown or Microsoft Word. Please make sure to include your full names and group number at the beginning of the document. Submit the questionnaire solution in PDF format by uploading it to the platform under the corresponding task section before the deadline.

Dataset

In this questionnaire, we will work with the following time series: Unemploy: Number of individuals registered as unemployed in SEPE offices since January 2000. <https://expinterweb.mites.gob.es/series/>. We will follow the questionnaire question by question.

1 - Loading the data

Instructions: load the file containing the series. Define the read data as an object of type ts (time series), specifying the origin and frequency of the series. Select the window of the series starting in 2000, and removing the four first years.

```
serie = window(ts(read.table("/Users/franz/Data_Science_R/Financial Econometrics/Financial Econometrics
                        dec = ",") / 1e6,
                start = 1996, freq = 12), # loading the data
                start = c(2000), end = c(2025, 12))
(round(serie, 3)) # rounding to third decimal
```

	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
## 2000	2.090	2.080	2.042	1.984	1.929	1.903	1.893	1.888	1.908	1.934	1.963	1.947
## 2001	2.017	1.993	1.981	1.910	1.898	1.843	1.836	1.879	1.889	1.941	1.986	1.989
## 2002	2.075	2.150	2.083	2.060	2.003	1.963	1.962	1.984	2.007	2.065	2.117	2.127
## 2003	2.185	2.180	2.163	2.104	2.036	2.020	1.996	2.017	2.040	2.097	2.143	2.181
## 2004	2.233	2.219	2.182	2.162	2.091	2.054	2.014	2.050	2.051	2.076	2.121	2.113
## 2005	2.177	2.165	2.145	2.096	2.007	1.975	1.989	2.019	2.013	2.053	2.096	2.103
## 2006	2.172	2.169	2.149	2.076	2.005	1.960	1.955	1.984	1.966	1.993	2.023	2.023

```
## 2007 2.083 2.075 2.059 2.023 1.973 1.966 1.970 2.028 2.017 2.049 2.094 2.130
## 2008 2.262 2.315 2.301 2.339 2.354 2.390 2.427 2.530 2.625 2.818 2.989 3.129
## 2009 3.328 3.482 3.605 3.645 3.620 3.565 3.544 3.629 3.709 3.808 3.869 3.924
## 2010 4.048 4.131 4.167 4.142 4.066 3.982 3.909 3.970 4.018 4.086 4.110 4.100
## 2011 4.231 4.299 4.334 4.269 4.190 4.122 4.080 4.131 4.227 4.361 4.420 4.422
## 2012 4.600 4.712 4.751 4.744 4.714 4.615 4.587 4.626 4.705 4.834 4.908 4.849
## 2013 4.981 5.040 5.035 4.989 4.891 4.764 4.699 4.699 4.724 4.811 4.809 4.701
## 2014 4.814 4.812 4.796 4.684 4.572 4.450 4.420 4.428 4.448 4.527 4.512 4.448
## 2015 4.526 4.512 4.452 4.333 4.215 4.120 4.046 4.068 4.094 4.176 4.149 4.094
## 2016 4.151 4.153 4.095 4.011 3.891 3.767 3.683 3.697 3.720 3.765 3.790 3.703
## 2017 3.760 3.751 3.702 3.573 3.461 3.363 3.336 3.382 3.410 3.467 3.474 3.413
## 2018 3.477 3.470 3.423 3.336 3.252 3.162 3.135 3.182 3.203 3.255 3.253 3.202
## 2019 3.286 3.289 3.255 3.164 3.079 3.016 3.011 3.066 3.080 3.178 3.198 3.164
## 2020 3.254 3.246 3.548 3.831 3.858 3.863 3.773 3.803 3.776 3.826 3.851 3.888
## 2021 3.964 4.009 3.950 3.911 3.781 3.614 3.416 3.334 3.258 3.257 3.183 3.106
## 2022 3.123 3.112 3.109 3.023 2.923 2.881 2.884 2.924 2.942 2.915 2.881 2.838
## 2023 2.908 2.911 2.862 2.788 2.739 2.689 2.678 2.703 2.722 2.759 2.735 2.707
## 2024 2.768 2.760 2.727 2.667 2.608 2.561 2.550 2.572 2.575 2.602 2.586 2.561
## 2025 2.599 2.593 2.580 2.513 2.455 2.406 2.405 2.427 2.422 2.444 2.425 2.409
```

2 - Graphical representation

Instructions: create a graphical representation of the time series. Describe the most relevant aspects observed at first glance.

```
plot(serie, main = "Unemployment Spain") # plotting the time series
abline(v = 2000:2026, col = 4, lty = 3)
```



Taking a look at this plot, we can spot constant level from 2000 to 2008, then an increasing trend after the 2008 crisis which reverts after 2013, with a big upwards spike during the Covid crisis. Moreover, there are some clear seasonal patterns. It is difficult to conclude from this plot alone whether the variance is increasing with the level.

3 - Making the time series stationary

Instructions: apply appropriate transformations to make the series stationary. Justify your decisions.

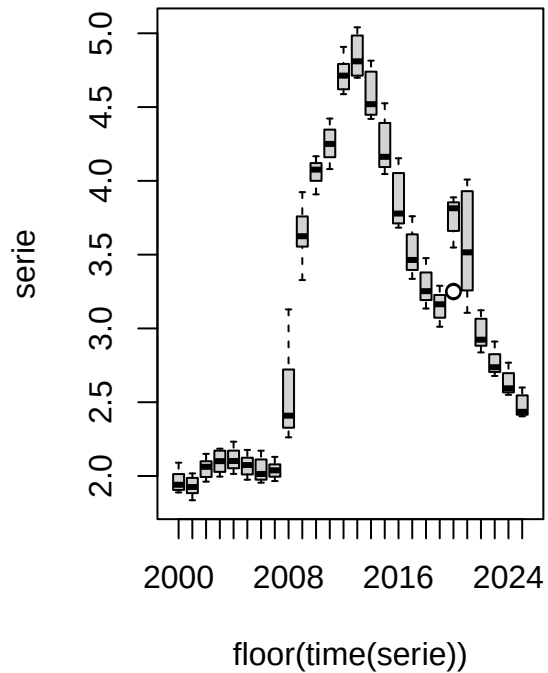
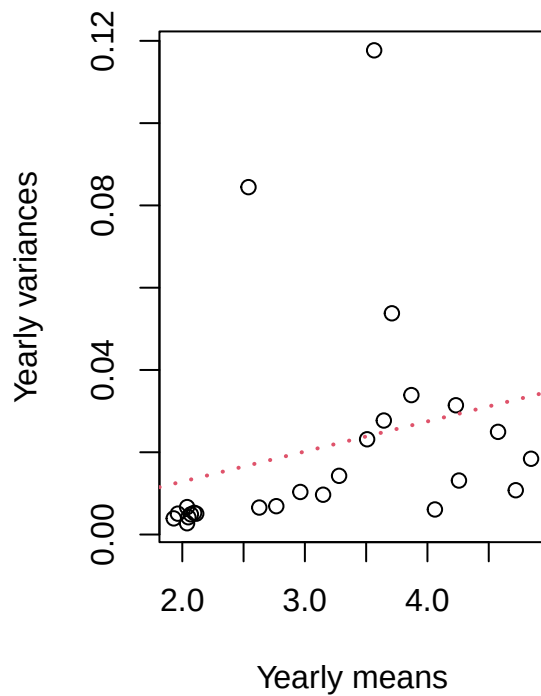
Analysis of the variance

First of all, we will identify if the variance of the time series increases as the level increases.

```
par(mfrow = c(1, 2))

# Mean-Variance plot
m <- apply(matrix(serie[1:(25 * 12)], nr = 12), 2, mean)
v <- apply(matrix(serie[1:(25 * 12)], nr = 12), 2, var)
plot(m, v, xlab = "Yearly means", ylab = "Yearly variances")
abline(lm(v ~ m), col = 2, lty = 3, lwd = 2)

# Boxplot by year
boxplot(serie ~ floor(time(serie)))
```



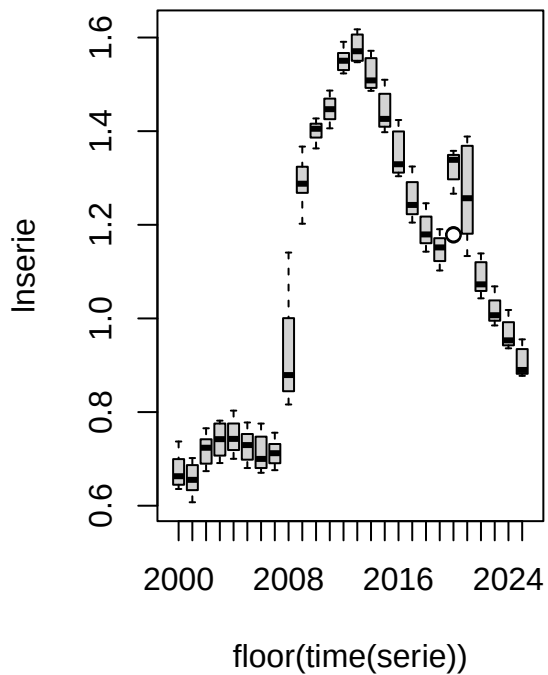
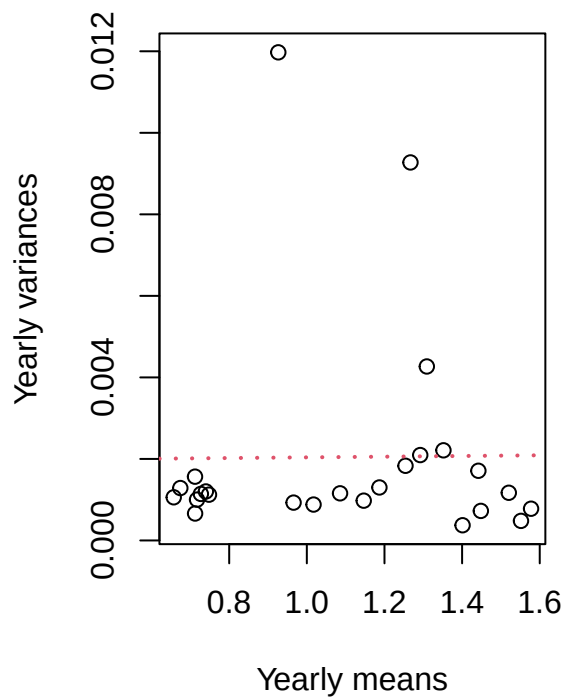
The positively sloped red line in the mean-variance plot and the increase in IQR range on higher levels both suggest a positive relationship between the variance and the level of the time series. Hence, we will log-transform the time series.

```
# Taking logs
lnserie <- log(serie)

par(mfrow = c(1, 2))

# Mean-Variance plot
m <- apply(matrix(lnserie[1:(25 * 12)]), nr = 12), 2, mean)
v <- apply(matrix(lnserie[1:(25 * 12)]), nr = 12), 2, var)
plot(m, v, xlab = "Yearly means", ylab = "Yearly variances")
abline(lm(v ~ m), col = 2, lty = 3, lwd = 2)

# Boxplot by year
boxplot(lnserie ~ floor(time(serie)))
```

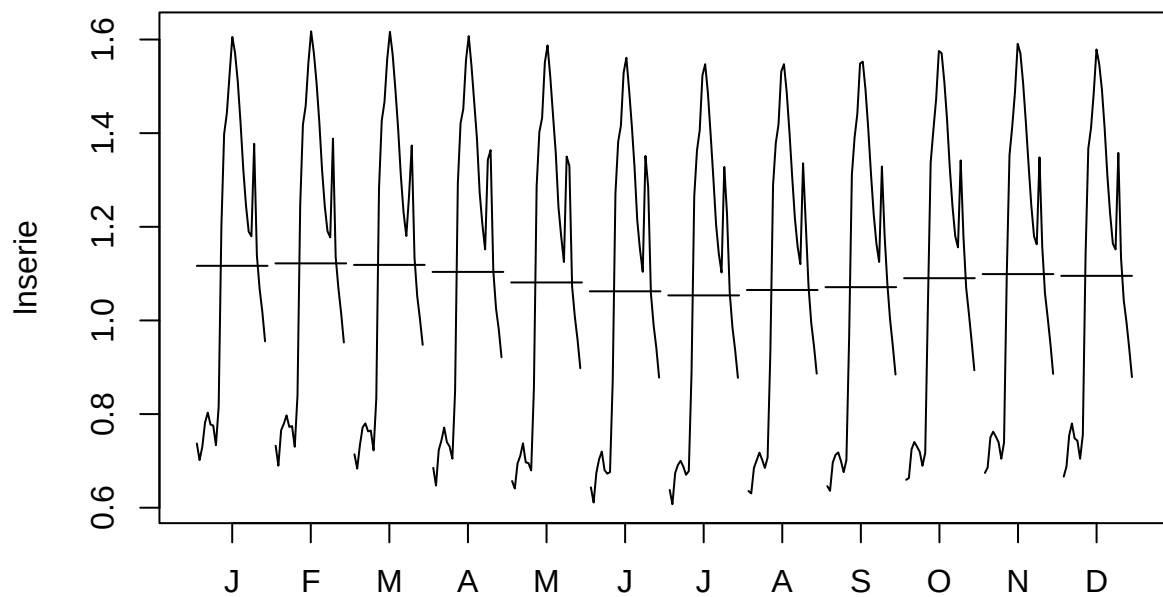


Indeed, the variance seems stable now, except for the years during the 2008 and 2020 shocks.

Analysis of the seasonality

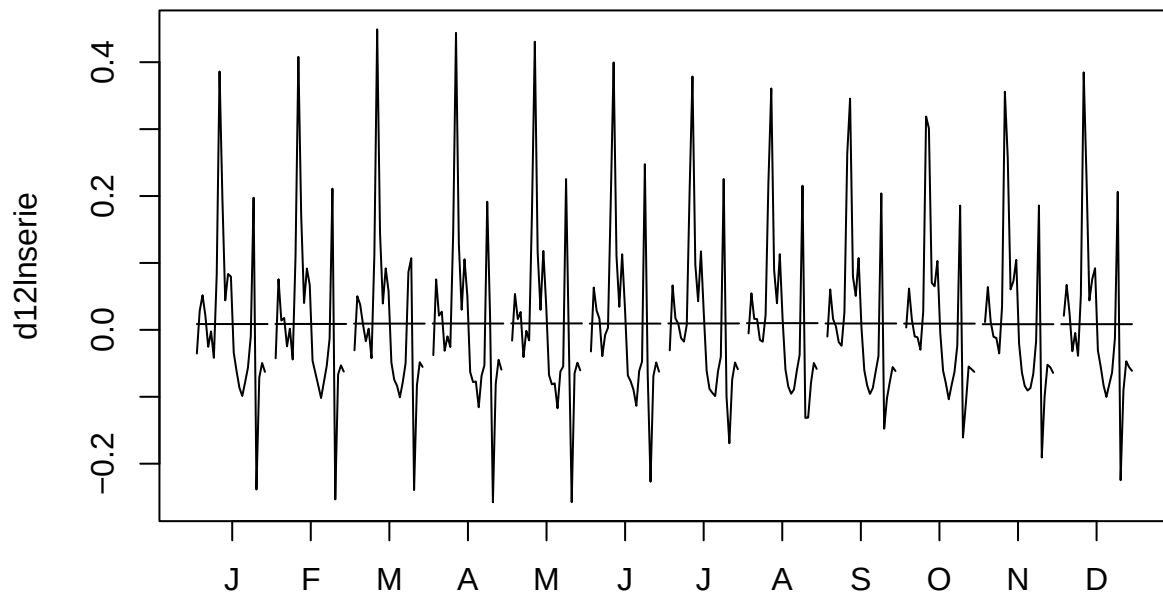
We'll plot a monthplot to decide whether to take seasonal differences.

```
monthplot(lnserie)
```



From the monthplot we can notice a slight decrease in unemployment during the summer months, hence there is seasonality. To fix this, we'll take seasonal differences.

```
d12lnserie <- diff(lnserie, 12)
monthplot(d12lnserie)
```

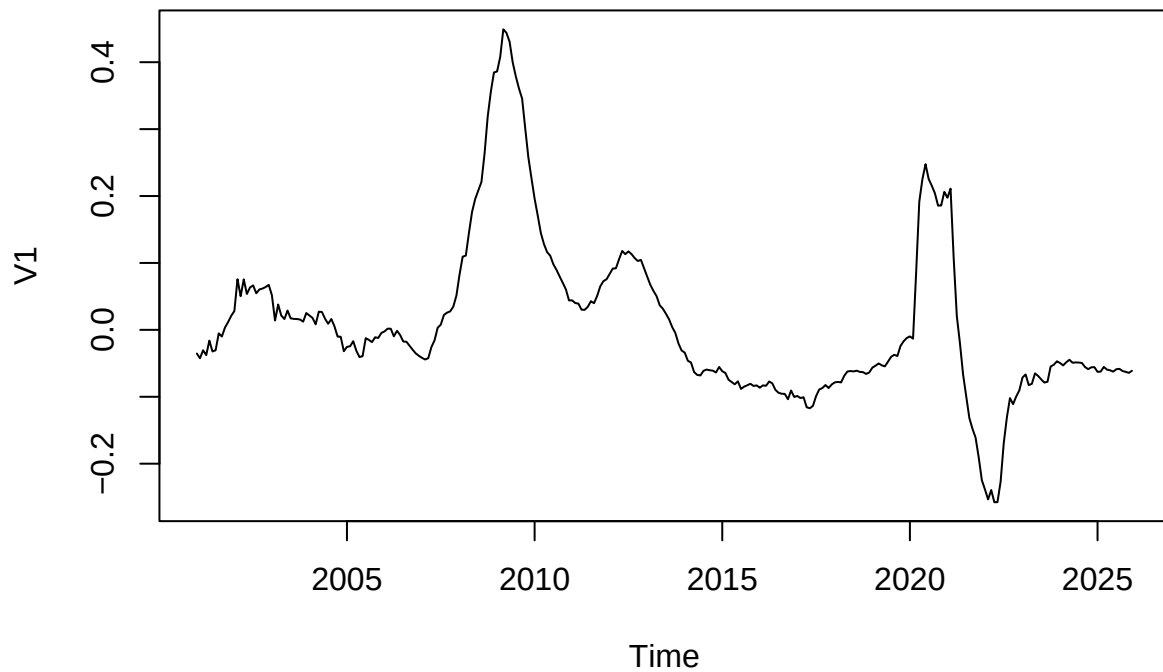


Now there are no seasonal patterns.

Analysis of the trend

The last step to achieve stationarity is to detrend the time series.

```
plot(d12lnserie)
```



There are some clear trends, especially during the crises, hence, we'll take simple differences until the variance of the transformed time series starts increasing again.

```
var(d12lnserie)
```

```
##           V1
## V1 0.01467464
```

```
d1d12lnserie <- diff(d12lnserie, 1)
var(d1d12lnserie)
```

```
##           V1
## V1 0.000349434
```

```
d1d1d12lnserie <- diff(d1d12lnserie, 1)
var(d1d1d12lnserie)
```

```
##           V1
## V1 0.000326053
```

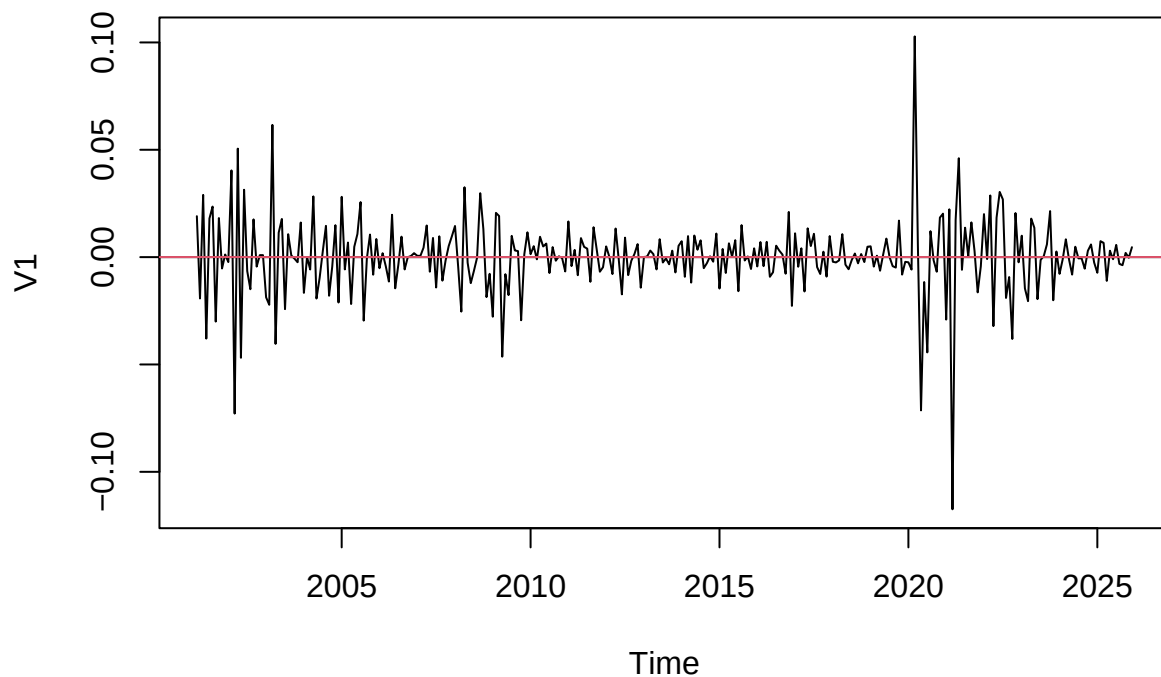
```
d1d1d1d12lnserie <- diff(d1d1d12lnserie, 1)
var(d1d1d1d12lnserie)
```



```
##          V1
## V1 0.0008836331
```

The time series with the lowest variance is the one with two simple differences.

```
plot(d1d1d12lnserie)
abline(h = 0)
abline(h = mean(d1d1d12lnserie), col = 2)
```



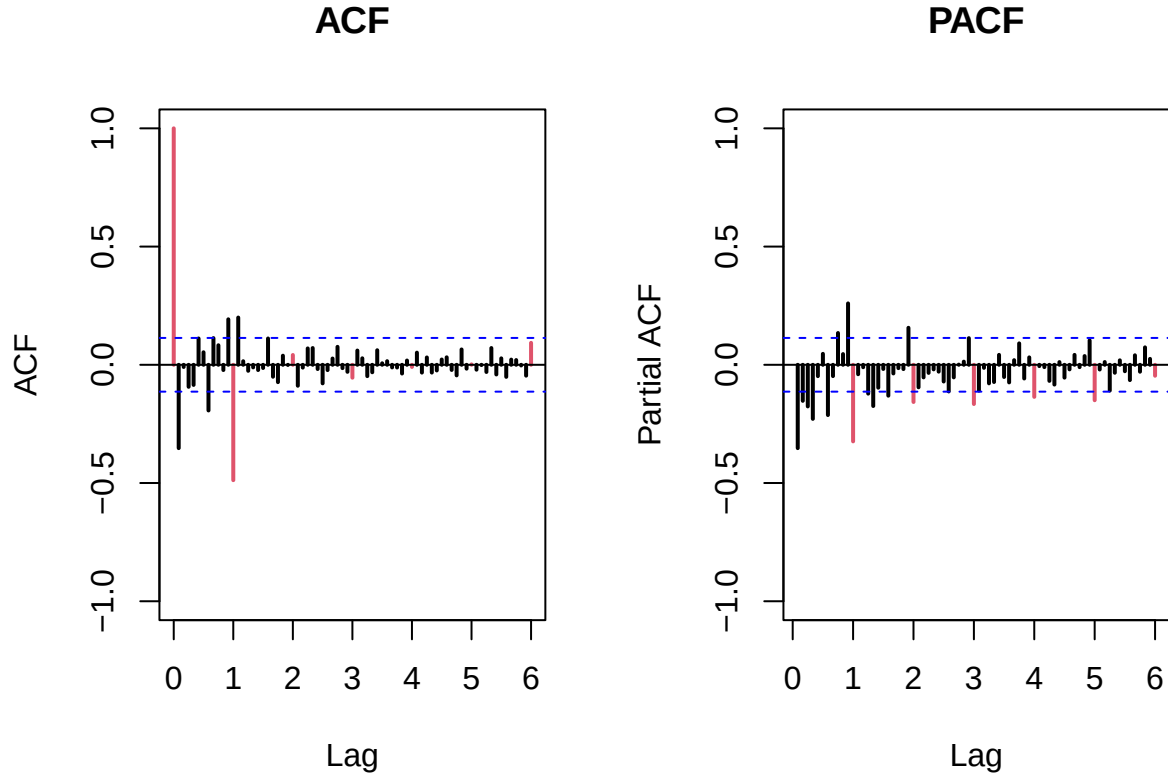
Note that from the plot there seems to be some volatility clustering. However, this seems to be induced mainly by outlier observations corresponding to the 2008 crisis and especially the Covid crisis. We'll rename the time series for simplicity.

```
ts <- d1d1d12lnserie
```

4 - Plotting ACF and PACF

Instructions: for the transformed series, plot the ACF and PACF.

```
par(mfrow = c(1, 2))
acf(ts, ylim = c(-1, 1), col = c(2, rep(1, 11)),
    lwd = 2, lag.max = 72, main = "ACF")
pacf(ts, ylim = c(-1, 1), col = c(rep(1, 11), 2),
    lwd = 2, lag.max = 72, main = "PACF")
```



Notice that the ACF drops rapidly from 1 to 0, hence the time series is indeed stationary.

5 - Model identification

Instructions: based on the sample ACF and PACF, propose at least two models for each series, justifying your proposals.

To identify possible models, we examine the ACF and PACF of the stationary series.

First, we determine the non-seasonal component of the model. To do this, we'll take into account only the lags previous to the seasonal period. Notice the ACF exceeds confidence intervals (CIs) significantly at lags 1 and 7, and then cuts off rapidly. On the other hand, the PACF decays relatively slowly and exceeds confidence intervals at higher lags. This pattern suggests an $ARIMA(0, 2, 1)$ or an $ARIMA(0, 2, 7)$ for the non-seasonal component of our model. However, since $ARIMA(0, 2, 7)$ is a fairly complex model, we'll also propose an $ARIMA(1, 2, 1)$ for the non-seasonal part. **Notice also that we could include the barely significant 8th lag in the ACF and propose an $ARIMA(0, 2, 8)$. However, to work with less models and because the ACF is barely significant at that lag, we propose only $ARIMA(0, 2, 7)$, which is the least complex model between the two.**

To determine the seasonal component of our model we analyse only the red lags. Again, the PACF exceeds the CIs even at high lags, while the ACF is inside the CIs at all lags after the first seasonal period. This patterns suggests a $SMA(1)$ at frequency 12.

Hence, the models we propose are:

- $SARIMA(0, 2, 1)(0, 1, 1)_{12}$
- $SARIMA(0, 2, 7)(0, 1, 1)_{12}$
- $SARIMA(1, 2, 1)(0, 1, 1)_{12}$

6 - Models estimation

Instructions: estimate the proposed models and verify the significance of the coefficients, ensuring that the residuals have an ACF compatible with white noise. If any coefficient is not significant, remove it from the model.

We will estimate all of the models and drop coefficients if they are insignificant (i.e. if $|t| < 2$)

SARIMA(0,2,1)(0,1,1)_{12} - Model 1

```
# Estimating SARIMA(0,2,1)(0,1,1){12}
mod1 <- arima(ts, order = c(0, 0, 1),
              seasonal = list(order = c(0, 0, 1), period = 12))
mod1

##
## Call:
## arima(x = ts, order = c(0, 0, 1), seasonal = list(order = c(0, 0, 1), period = 12))
##
## Coefficients:
##          ma1      sma1  intercept
##      -0.5276  -0.8517      0e+00
## s.e.    0.0785   0.0413      1e-04
##
## sigma^2 estimated as 0.0001595:  log likelihood = 872.01,  aic = -1736.03
```

```
# Computing t-values
coef <- coef(mod1)
se <- sqrt(diag(vcov(mod1)))
tvals <- coef / se
cbind("Coef" = coef, "SE" = se, "t-value" = tvals)
```

	Coef	SE	t-value
ma1	-5.276410e-01	7.854514e-02	-6.7176792
sma1	-8.516999e-01	4.133811e-02	-20.6032608
intercept	-9.139582e-06	9.037688e-05	-0.1011274

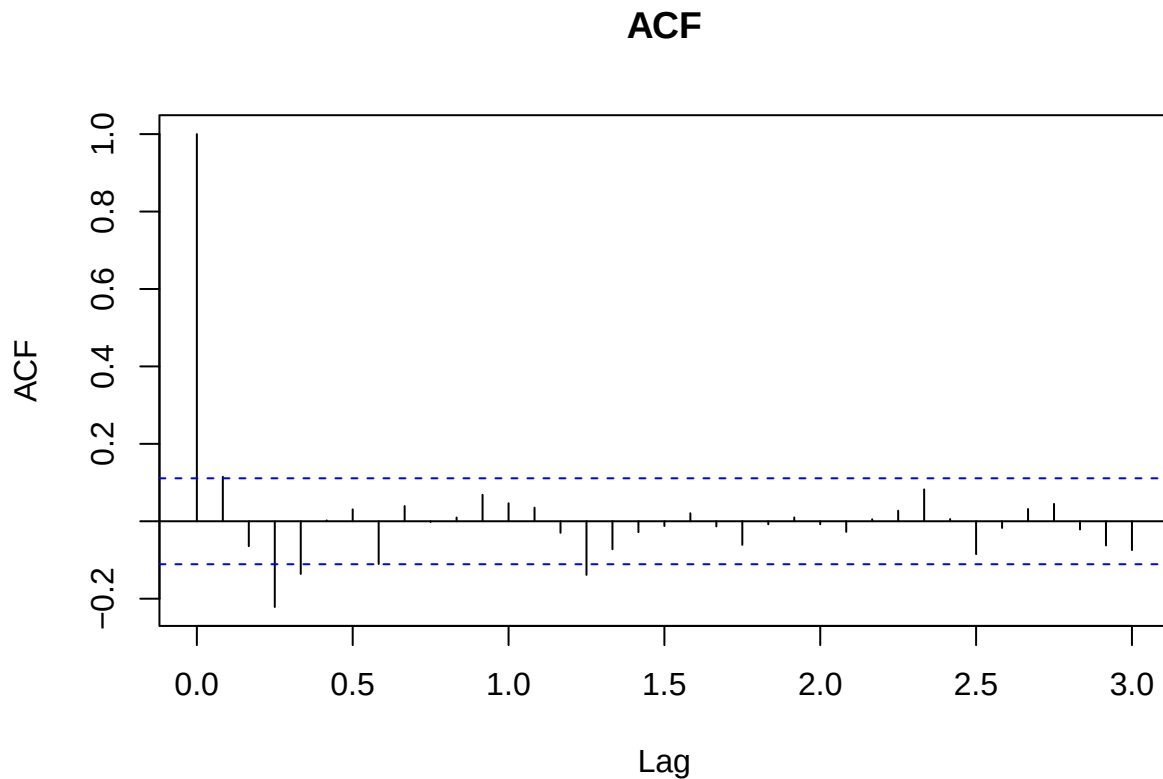
Since, $|t| > 2$ for all of the estimated coefficient, all of them are significant. However, since $|t| < 2$ for the intercept (i.e. the intercept is not significant), we re-estimate the model on the non-differenced series.

```
# Estimating SARIMA(0,2,1)(0,1,1){12}
mod1 <- arima(lnserie, order = c(0, 2, 1),
              seasonal = list(order = c(0, 1, 1), period = 12))
mod1

##
## Call:
## arima(x = lnserie, order = c(0, 2, 1), seasonal = list(order = c(0, 1, 1), period = 12))
##
## Coefficients:
##          ma1      sma1
```

```
##      -0.5275  -0.8517
## s.e.   0.0786   0.0413
##
## sigma^2 estimated as 0.0001595:  log likelihood = 869.12,  aic = -1732.23
```

```
acf(residuals(mod1), main = "ACF", lag.max = 36)
```



The model's residuals seem to behave like a white noise, dropping to 0 after the first lag and with few lags exceeding the CIs.

SARIMA(0,2,7)(0,1,1)_{12} - Model 2

```
# Estimating SARIMA(0,2,7)(0,1,1){12}
mod2 <- arima(ts, order = c(0, 0, 7),
              seasonal = list(order = c(0, 0, 1), period = 12))
mod2
```

```
##
## Call:
## arima(x = ts, order = c(0, 0, 7), seasonal = list(order = c(0, 0, 1), period = 12))
##
## Coefficients:
##          ma1          ma2          ma3          ma4          ma5          ma6          ma7          sma1
```

```
##      -0.4300 -0.1159 -0.2267 -0.0387  0.0777  0.0368 -0.1528 -0.8886
## s.e.   0.0586  0.0638  0.0627  0.0599  0.0636  0.0670  0.0676  0.0417
##      intercept
##              0e+00
## s.e.        1e-04
##
## sigma^2 estimated as 0.0001413:  log likelihood = 887.69,  aic = -1755.38
```

```
coef <- coef(mod2)
se <- sqrt(diag(vcov(mod2)))
tvals <- coef / se
cbind("Coef" = coef, "SE" = se, "t-value" = tvals)
```

```
##              Coef      SE      t-value
## ma1      -4.300320e-01 0.0586408660 -7.3333155
## ma2      -1.158864e-01 0.0637666695 -1.8173499
## ma3      -2.266924e-01 0.0627067310 -3.6151209
## ma4      -3.865195e-02 0.0598910116 -0.6453714
## ma5       7.770758e-02 0.0636404886  1.2210399
## ma6       3.684619e-02 0.0670424341  0.5495951
## ma7      -1.528474e-01 0.0676370205 -2.2598185
## sma1      -8.886367e-01 0.0416627608 -21.3292813
## intercept -7.293389e-06 0.0000612207 -0.1191327
```

For this model, we remove all of the coefficients with $|t| < 2$, which are not statistically significant. Hence, we remove:

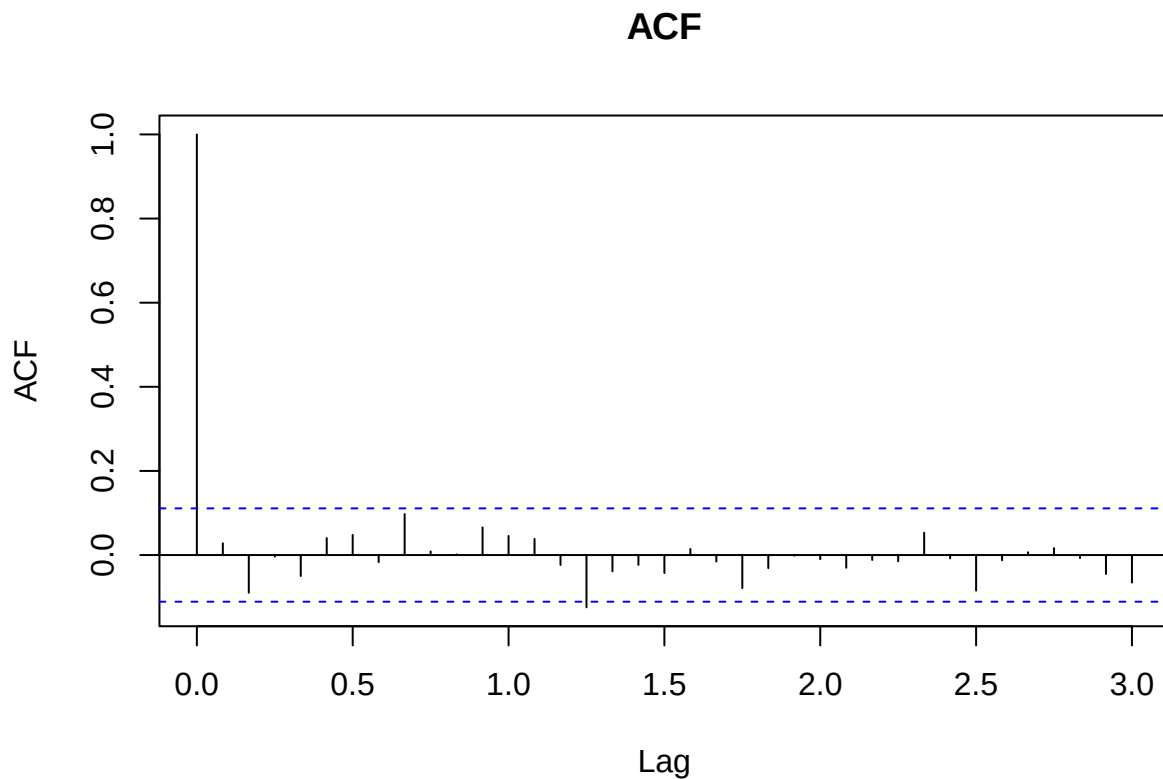
- ma2
- ma4
- ma5
- ma6

Moreover, since $|t| < 2$ for the intercept (i.e. the intercept is not significant), we re-estimate the model on the non-differenced series.

```
# Estimating SARIMA(0,2,7)(0,1,1)12
mod2 <- arima(lnserie, order = c(0, 2, 7),
              seasonal = list(order = c(0, 1, 1), period = 12),
              fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
mod2
```

```
##
## Call:
## arima(x = lnserie, order = c(0, 2, 7), seasonal = list(order = c(0, 1, 1), period = 12),
##      fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
##
## Coefficients:
##      ma1  ma2      ma3  ma4  ma5  ma6      ma7      sma1
##      -0.4731   0 -0.2607   0   0   0 -0.1298 -0.8840
## s.e.   0.0539   0  0.0517   0   0   0  0.0627  0.0418
##
## sigma^2 estimated as 0.0001437:  log likelihood = 882.43,  aic = -1754.86
```

```
acf(residuals(mod2), main = "ACF", lag.max = 36)
```



Again, the model's residual behave as white noise.

SARIMA(1,2,1)(0,1,1)_{12} - Model 3

```
# Estimating SARIMA(1,2,1)(0,1,1){12}
mod3 <- arima(ts, order = c(1, 0, 1),
              seasonal = list(order = c(0, 0, 1), period = 12))
mod3

##
## Call:
## arima(x = ts, order = c(1, 0, 1), seasonal = list(order = c(0, 0, 1), period = 12))
##
## Coefficients:
##          ar1          ma1          sma1  intercept
##          0.4421   -0.8772   -0.8859           0e+00
## s.e.    0.0928    0.0600    0.0412           1e-04
##
## sigma^2 estimated as 0.0001473:  log likelihood = 882.06,  aic = -1754.12
```

```
coef <- coef(mod3)
se <- sqrt(diag(vcov(mod3)))
tvals <- coef / se
cbind("Coef" = coef, "SE" = se, "t-value" = tvals)
```

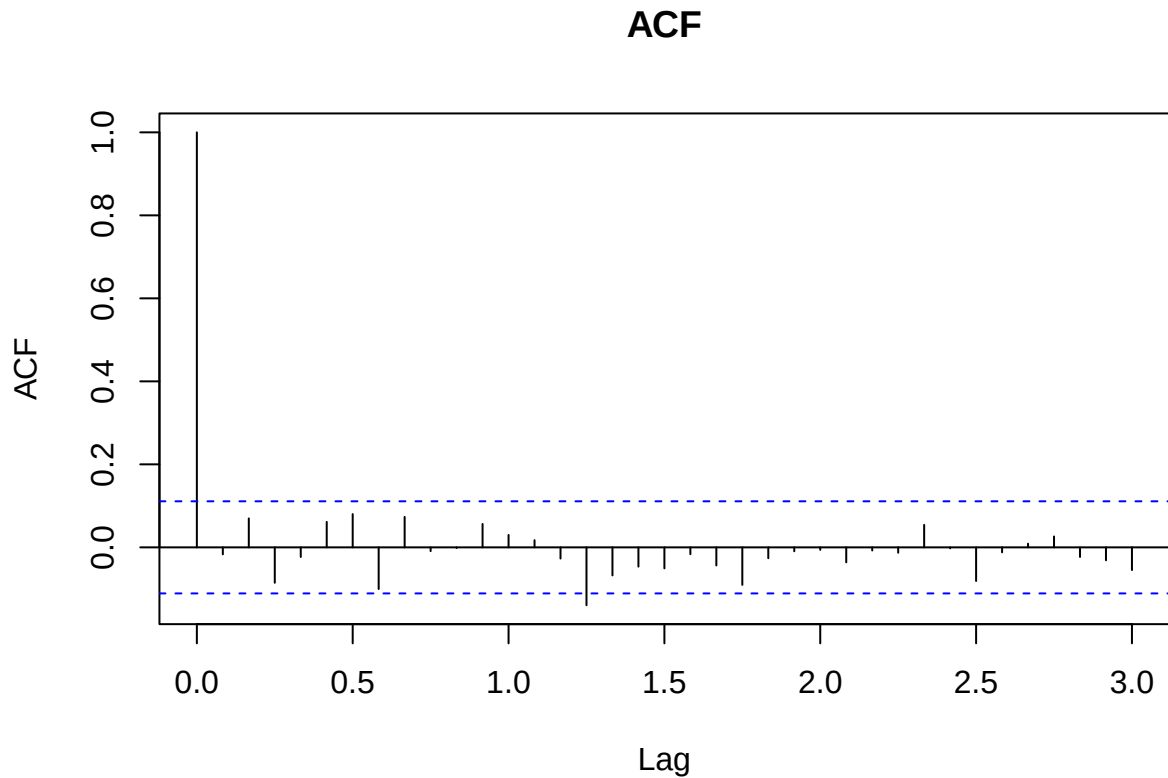
```
##              Coef          SE      t-value
## ar1          4.421447e-01 9.278991e-02   4.7650087
## ma1          -8.772336e-01 6.003850e-02 -14.6111856
## sma1         -8.859104e-01 4.119602e-02 -21.5047548
## intercept   -7.757699e-06 6.443336e-05  -0.1203988
```

All of the coefficients are significant, hence we do not drop them. However, since $|t| < 2$ for the intercept, we re-estimate the model on the non-differenced series.

```
# Estimating SARIMA(1,2,1)(0,1,1)12
mod3 <- arima(lnserie, order = c(1, 2, 1),
              seasonal = list(order = c(0, 1, 1), period = 12))
mod3
```

```
##
## Call:
## arima(x = lnserie, order = c(1, 2, 1), seasonal = list(order = c(0, 1, 1), period = 12))
##
## Coefficients:
##          ar1          ma1          sma1
##          0.4413   -0.8762   -0.8852
## s.e.  0.0924    0.0596    0.0410
##
## sigma^2 estimated as 0.0001474:  log likelihood = 879.14,  aic = -1750.28
```

```
acf(residuals(mod3), main = "ACF", lag.max = 36)
```



Model's residuals behave as a white noise.

Residual analysis and performance metrics

Instructions: for each model, perform the residuals analysis and calculate the metrics to compare models

A - Model roots

Instructions: using the characteristic polynomials of the AR and MA components, calculate the modulus of their roots. Is the estimated model causal? Is it invertible?

For a model to be causal and invertible the roots of the characteristic polynomial of the $MA(\infty)$ and the $AR(\infty)$ must be outside of the unit circle.

Model 1

We compute the roots for model (only the MA polynomial).

```
Mod(polyroot(c(1, mod1$model$theta)))
```

```
## [1] 1.013468 1.013468 1.013468 1.013468 1.013468 1.013468 1.013468 1.013468
## [9] 1.013468 1.013468 1.013468 1.013468 1.895708
```

All roots are outside the unit circle, hence the model is both causal and invertible.

Model 2

We compute the roots for model 2 (only the MA polynomial).

```
Mod(polyroot(c(1, mod2$model$theta)))
```

```
## [1] 1.010325 1.010325 1.010325 1.010325 1.010325 1.010325 1.010325 1.010325 1.010325
## [9] 1.010325 1.010325 1.010325 1.057277 1.298849 1.473253 1.298849 1.010325
## [17] 1.473253 1.410719 1.410719
```

All roots are outside the unit circle, hence the model is both causal and invertible.

Model 3

We compute the roots for model 3.

```
Mod(polyroot(c(1, -mod3$model$phi))) # MA
```

```
## [1] 2.266164
```

```
Mod(polyroot(c(1, mod3$model$theta))) # AR
```

```
## [1] 1.010211 1.010211 1.010211 1.010211 1.010211 1.010211 1.010211 1.010211 1.010211
## [9] 1.010211 1.010211 1.010211 1.010211 1.141325
```

All roots are outside the unit circle, hence the model is both causal and invertible.

B - Residuals plot

Instructions: plot the residuals and the square root of the absolute residuals with a smoothing adjustment. Can we assume that variance is constant?

For each model we will plot the time series of the residuals and the square root of absolute residuals with smoothing. If residuals are homoskedastic, there shouldn't be volatility clustering in the first plot and the red line should be flat in the second one.

Model 1

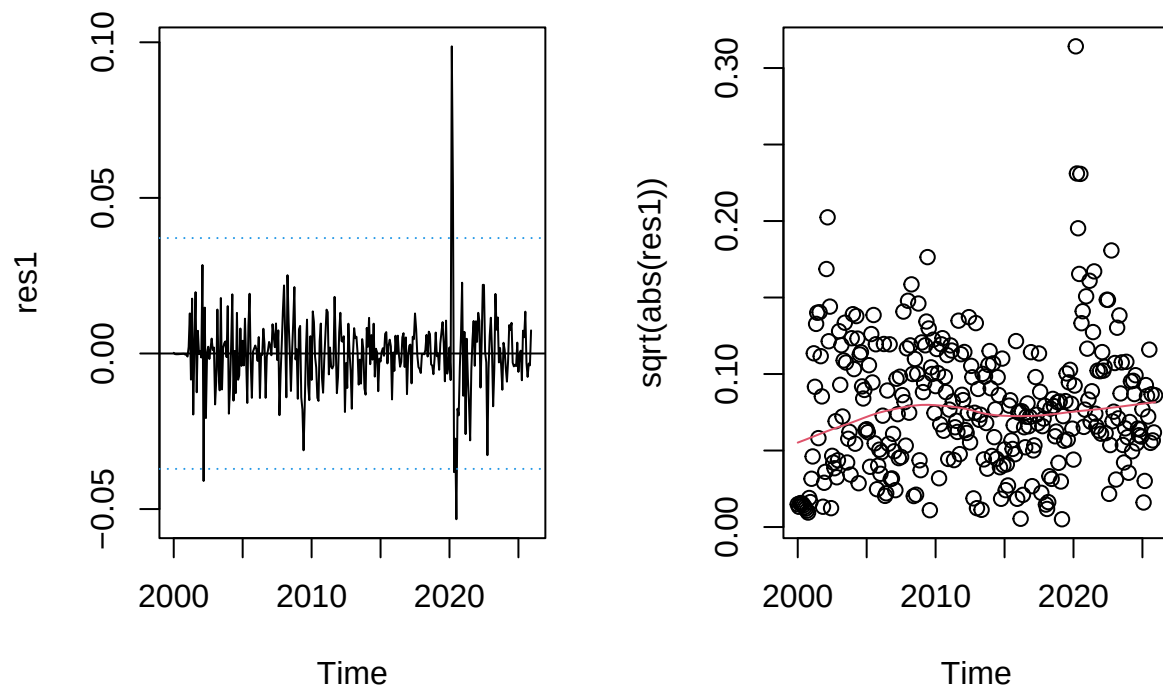
We compute and plot the residuals for model 1.

```
# Computing residuals
res1 <- residuals(mod1)

par(mfrow = c(1, 2))

# Residuals plot
plot(res1)
abline(h = 0)
abline(h = c(-3 * sd(res1), 3 * sd(res1)), lty = 3, col = 4)

# Plot of the square root of absolute residuals with smoothing
scatter.smooth(sqrt(abs(res1)), lpars = list(col = 2))
```



There is some volatility clustering (especially during the 2020 crisis) and the red line in the right plot is not completely flat, hence there residuals don't seem to be homoskedastic. However, this is probably caused by outliers, which treatment hasn't been covered in class.

Model 2

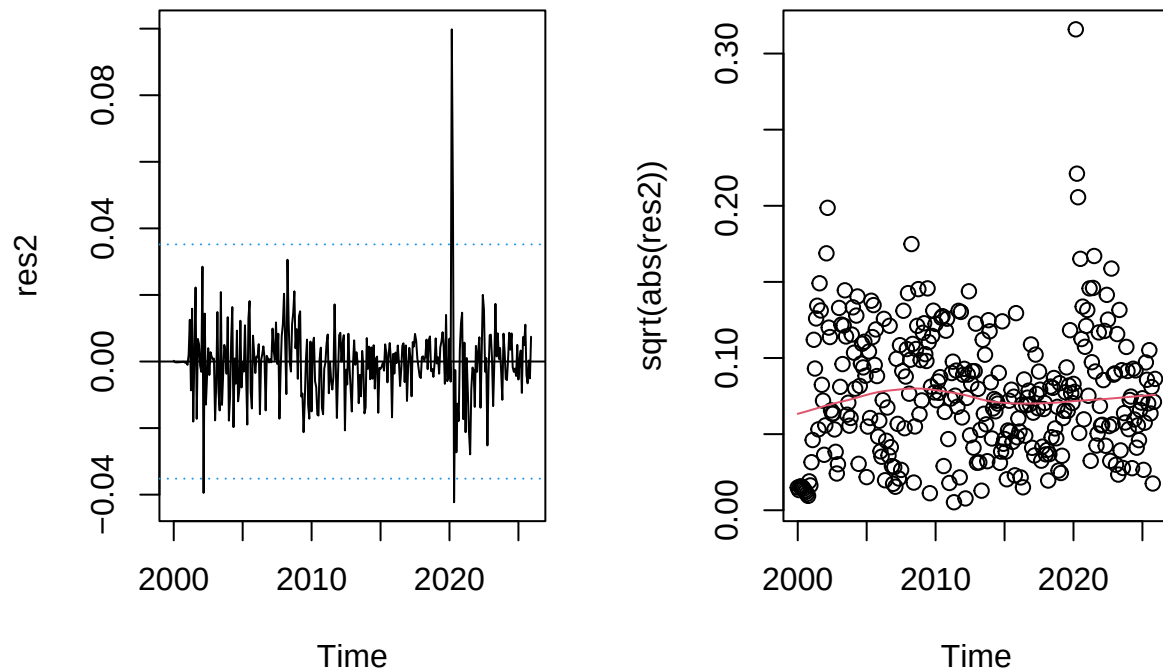
We compute and plot the residuals for model 2.

```
# Computing residuals
res2 <- residuals(mod2)

par(mfrow = c(1, 2))

# Residuals plot
plot(res2)
abline(h = 0)
abline(h = c(-3 * sd(res2), 3 * sd(res2)), lty = 3, col = 4)

# Plot of the square root of absolute residuals with smoothing
scatter.smooth(sqrt(abs(res2)), lpars = list(col = 2))
```



The two plots look very similar to the ones of model 1, hence the interpretation is the same.

Model 3

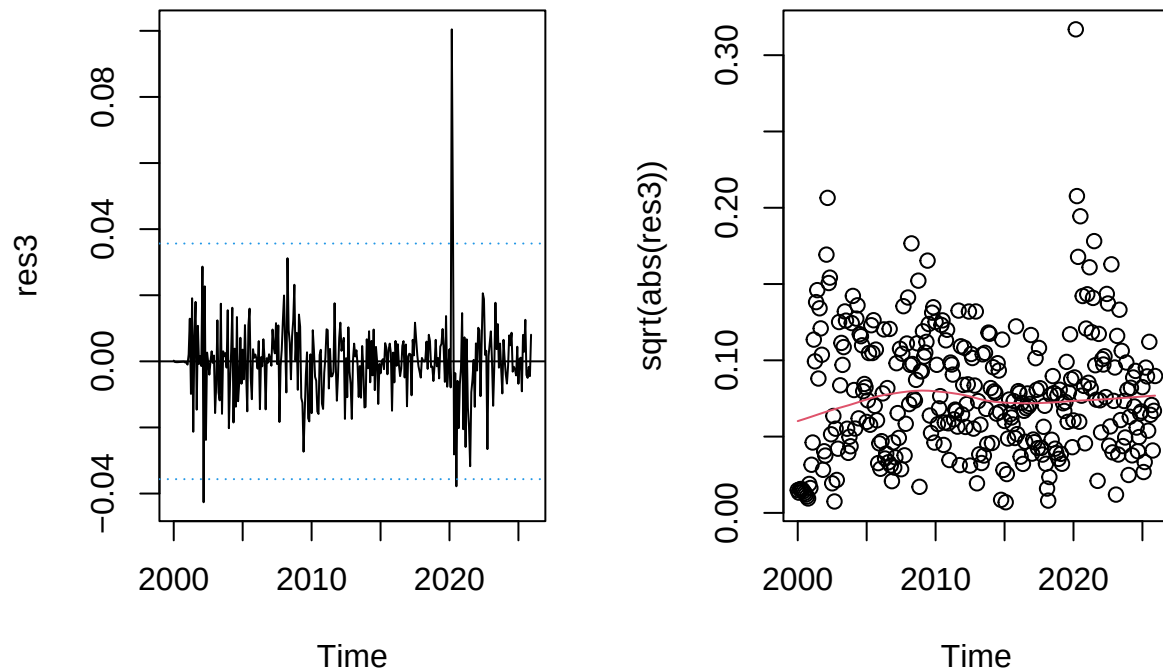
We compute and plot the residuals for model 3.

```
# Computing residuals
res3 <- residuals(mod3)

par(mfrow = c(1, 2))

# Residuals plot
plot(res3)
abline(h = 0)
abline(h = c(-3 * sd(res3), 3 * sd(res3)), lty = 3, col = 4)

# Plot of the square root of absolute residuals with smoothing
scatter.smooth(sqrt(abs(res3)), lpars = list(col = 2))
```



Again, the interpretation is the same as the one for the other two models.

C - Normality plot and Shapiro-Wilk test

Instructions: generate the normality plot and the histogram with the normal curve superimposed. Apply the Shapiro-Wilk test to the residuals. Can we assume that residuals follow a normal distribution?

We assume that the residuals are normally distributed if the plot of their distribution resembles the one of the normal distribution (red line in the following plots). Moreover, if they are normally distributed, the Shapiro-Wilk test should return a p-value greater than 0.05, meaning that we cannot reject the null hypothesis of normality at the 95% significance level.

Model 1

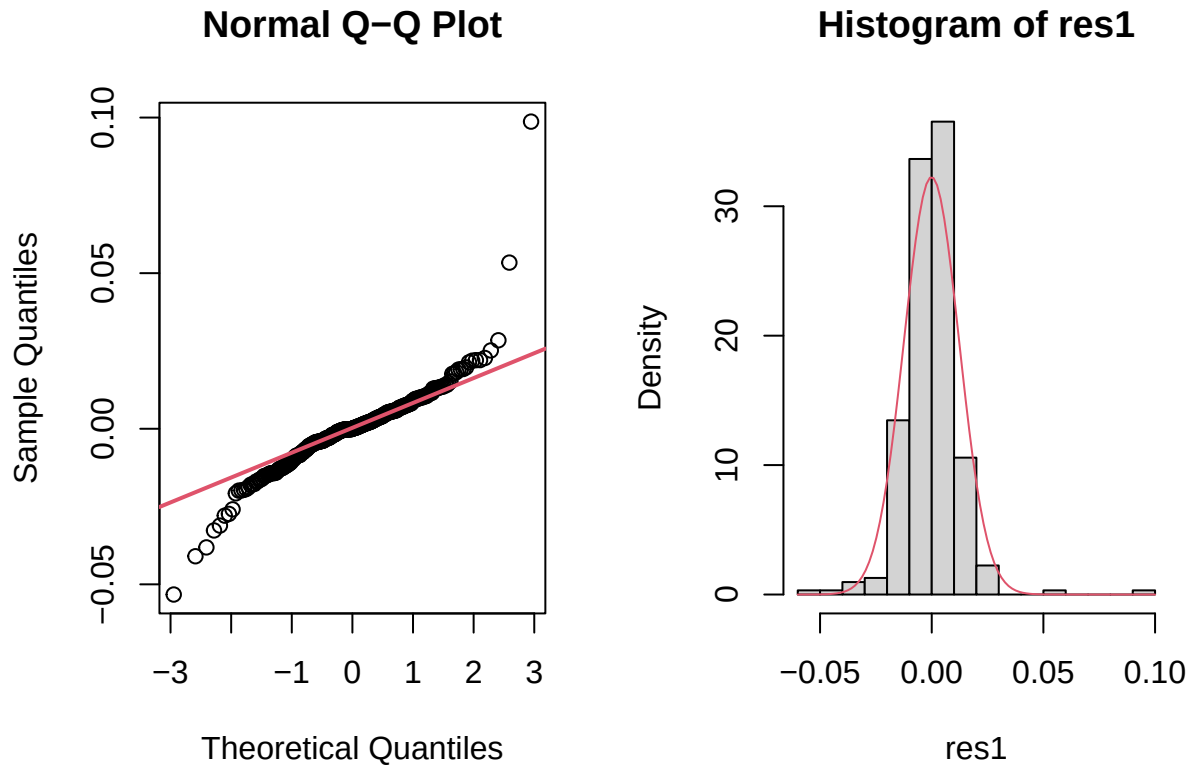
We plot the distribution of the residuals of model 1 against the normal distribution.

```
par(mfrow = c(1, 2))

# Normality plot
qqnorm(res1)
qqline(res1, col = 2, lwd = 2)

# Histogram with superimposed normal curve
hist(res1, breaks = 20, freq = FALSE)
```

```
curve(dnorm(x, mean = mean(res1), sd = sd(res1)), col=2,
add = T)
```



The tails of the residuals are fatter compared to the one of the normal distribution. However, this, again, is mainly due to the presence of outliers.

```
# Shapiro-Wilk test
shapiro.test(res1)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  res1
## W = 0.86912, p-value = 1.302e-15
```

Since the $p - value < 0.05$ the Shapiro-Wilk test concludes that the residuals are not normally distributed. However, this results is probably biased because of the outliers.

Model 2

We plot the distribution of the residuals of model 2 against the normal distribution.

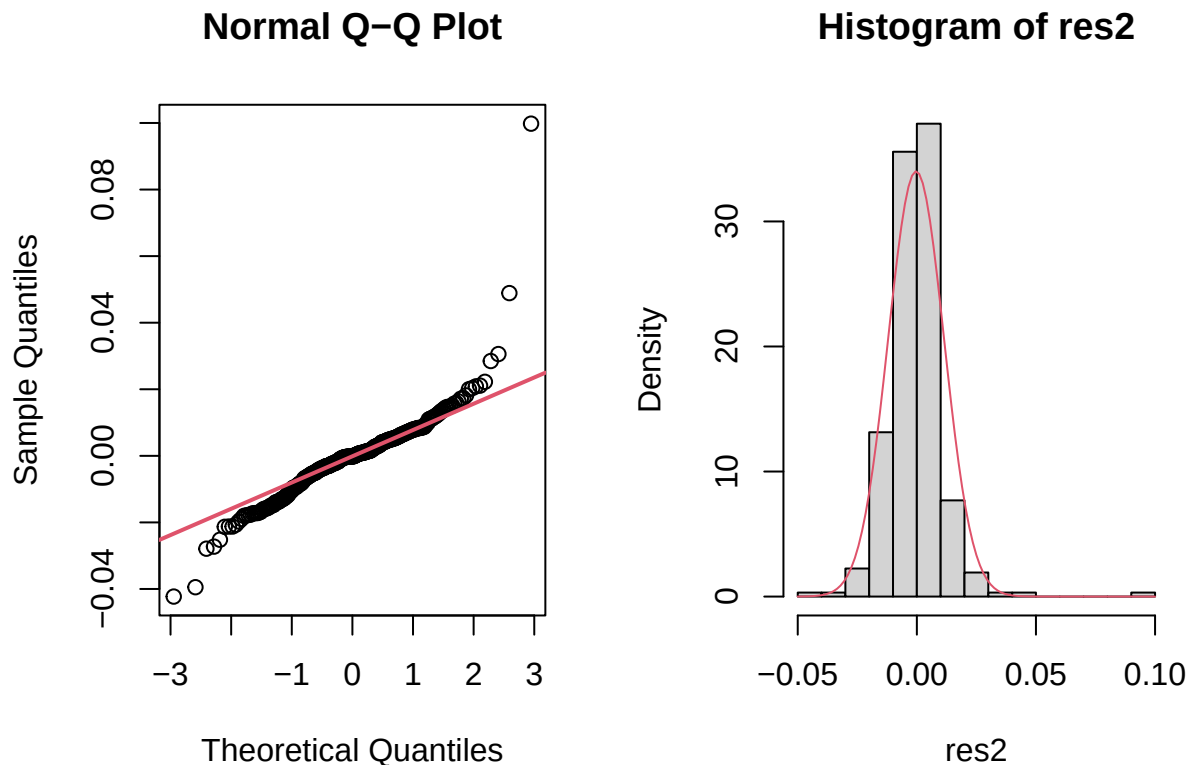
```

par(mfrow = c(1, 2))

# Normality plot
qqnorm(res2)
qqline(res2, col = 2, lwd = 2)

# Histogram with superimposed normal curve
hist(res2, breaks = 20, freq = FALSE)
curve(dnorm(x, mean = mean(res2), sd = sd(res2)), col=2,
add = T)

```



The two plots look very similar to the ones of model 1, hence the interpretation is the same.

```

# Shapiro-Wilk test
shapiro.test(res2)

```

```

##
##  Shapiro-Wilk normality test
##
## data:  res2
## W = 0.86155, p-value = 4.462e-16

```

Again, the result is similar to the one of model 1, so the interpretation is the same.

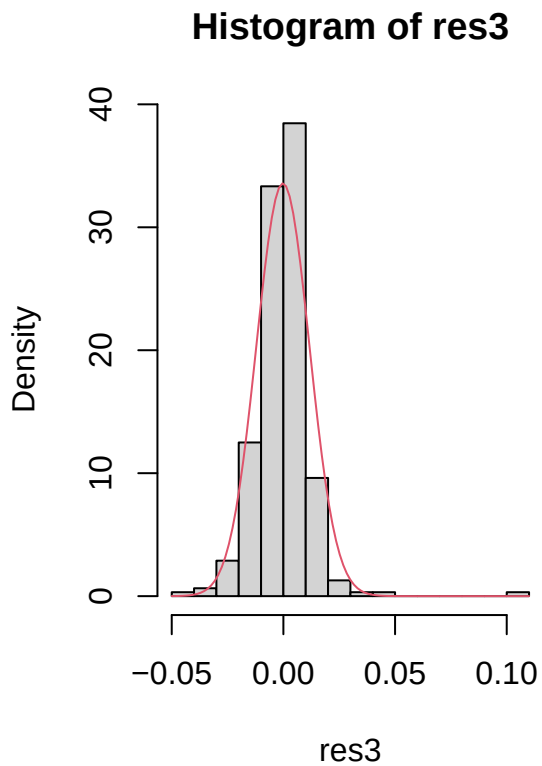
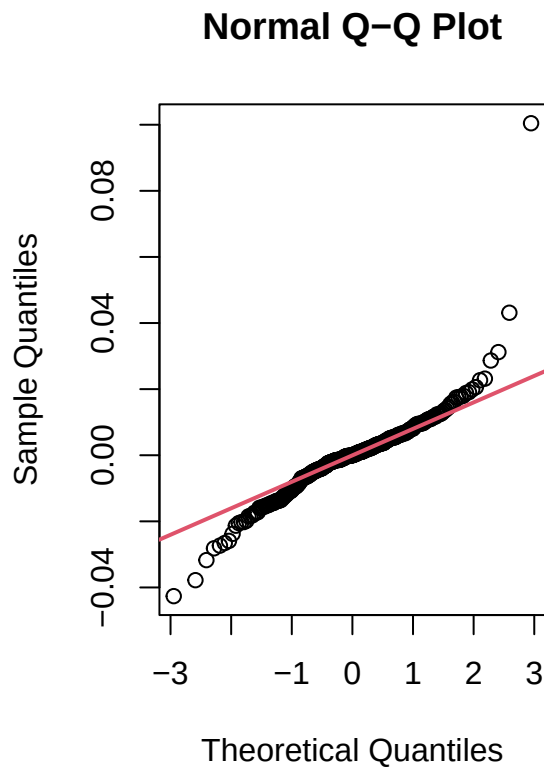
Model 3

We plot the distribution of the residuals of model 3 against the normal distribution.

```
par(mfrow = c(1, 2))

# Normality plot
qqnorm(res3)
qqline(res3, col = 2, lwd = 2)

# Histogram with superimposed normal curve
hist(res3, breaks = 20, freq = FALSE)
curve(dnorm(x, mean = mean(res3), sd = sd(res3)), col=2,
add = T)
```



```
# Shapiro-Wilk test
shapiro.test(res3)
```

```
##
##  Shapiro-Wilk normality test
##
## data:  res3
## W = 0.86896, p-value = 1.272e-15
```

Same interpretation as the other models.

D - Residuals ACF and PACF and Ljung-Box test

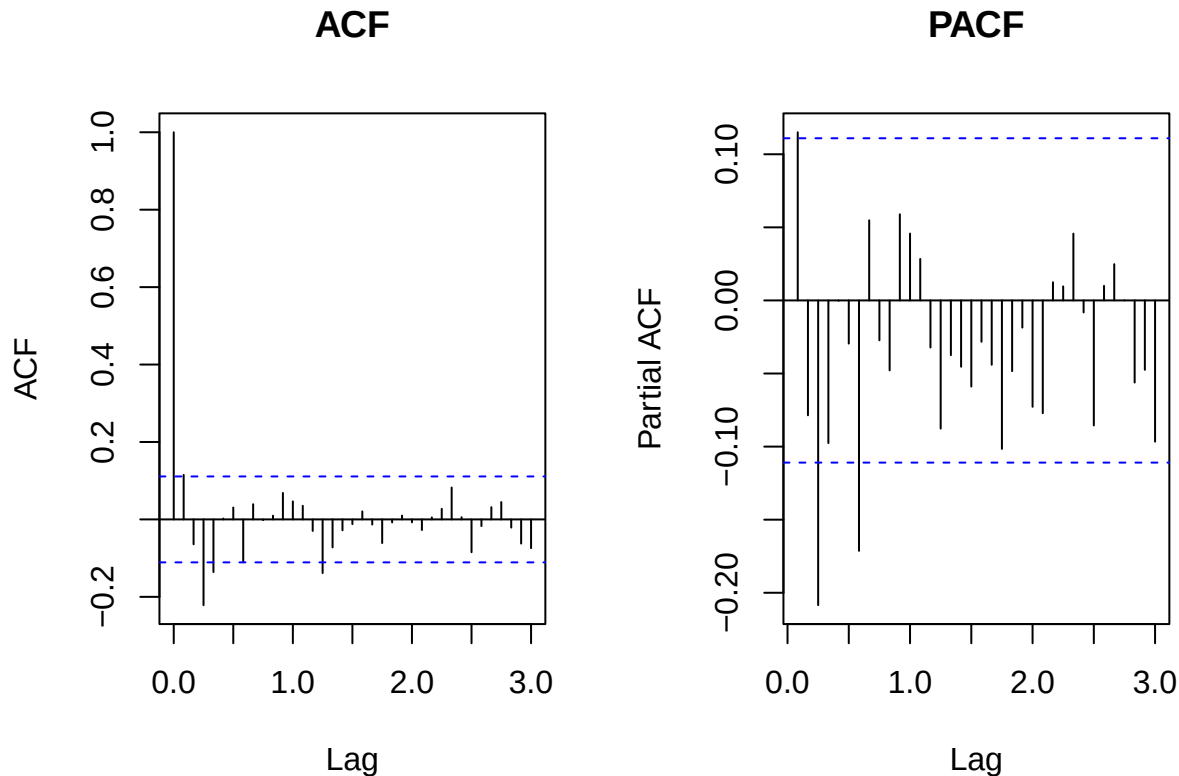
Instructions: plot the ACF and PACF of residuals and display p-values from the Ljung-Box test. Can we assume residuals are independent?

Looking at the ACF and PACF, residuals are independent if 95% of lags are inside the 95% CIs. However, the analysis of the ACF and PACF is useful to check the significance only of individual autocorrelations, ignoring that the configuration of a subset or lags might be jointly significant. To test this, we use the Ljung-Box test, which tests the hypothesis that the first k autocorrelations of the residuals are jointly 0. Hence, for there to be independence between residuals, $p\text{-value} > 0.05$ for each lag.

Model 1

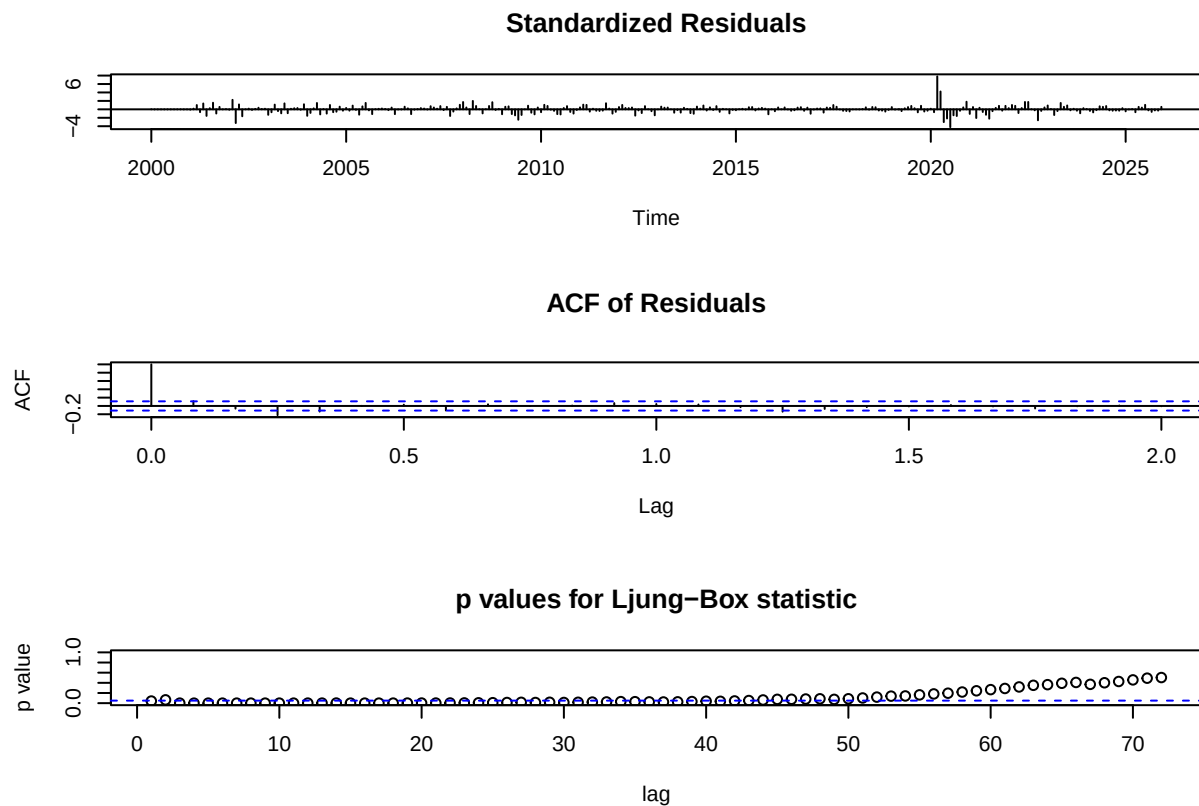
We plot the ACF and PACF of the residuals of model 1.

```
par(mfrow = c(1, 2))  
  
# ACF and PACF  
acf(res1, main = "ACF", lag.max = 36)  
pacf(res1, main = "PACF", lag.max = 36)
```



In both the ACF and PACF some lags exceed the CIs, but 1 out of 20 residuals is an acceptable rate given that we are considering 95% CIs. Hence, the residuals appear to be independent. The Ljung-Box test will provide better insight into this matter.


```
# Ljung-Box test
tsdiag(mod1, gof.lag = 72)
```



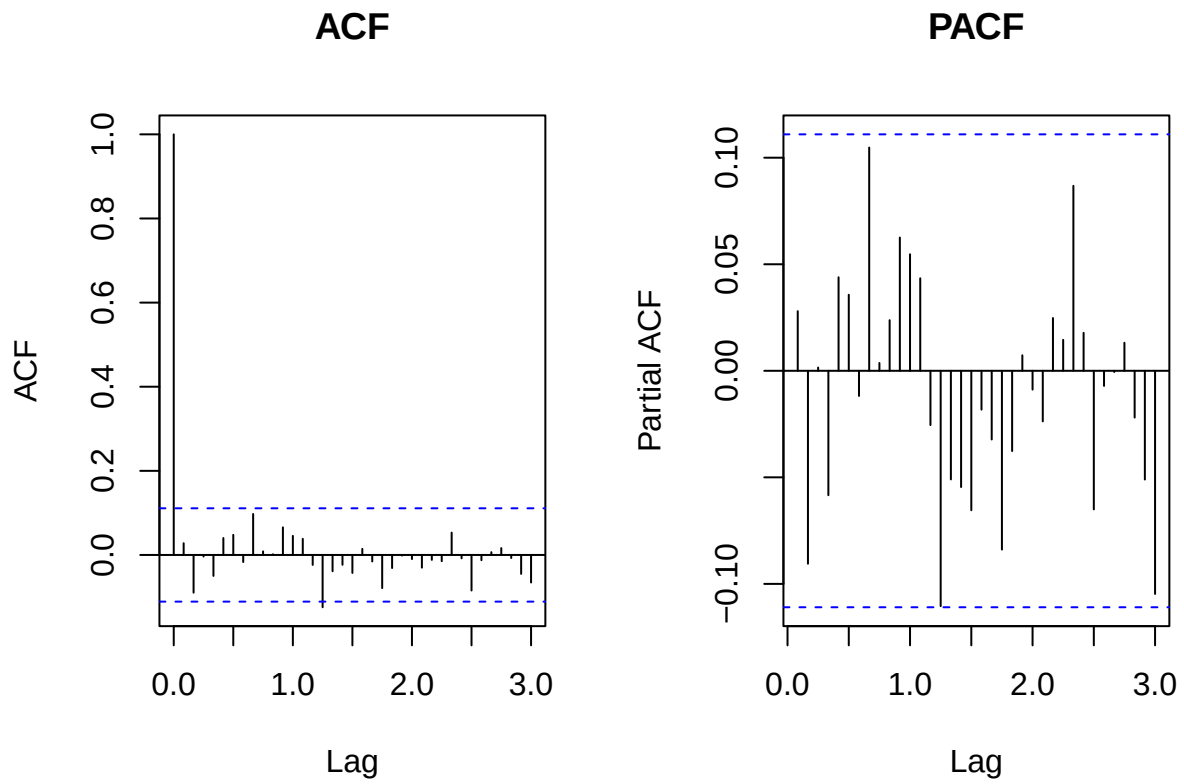
For a lot of lags $p - value < 0.05$, hence there is dependence between residuals. Therefore, we will not use this model for prediction.

Model 2

We plot the ACF and PACF of the residuals of model 2.

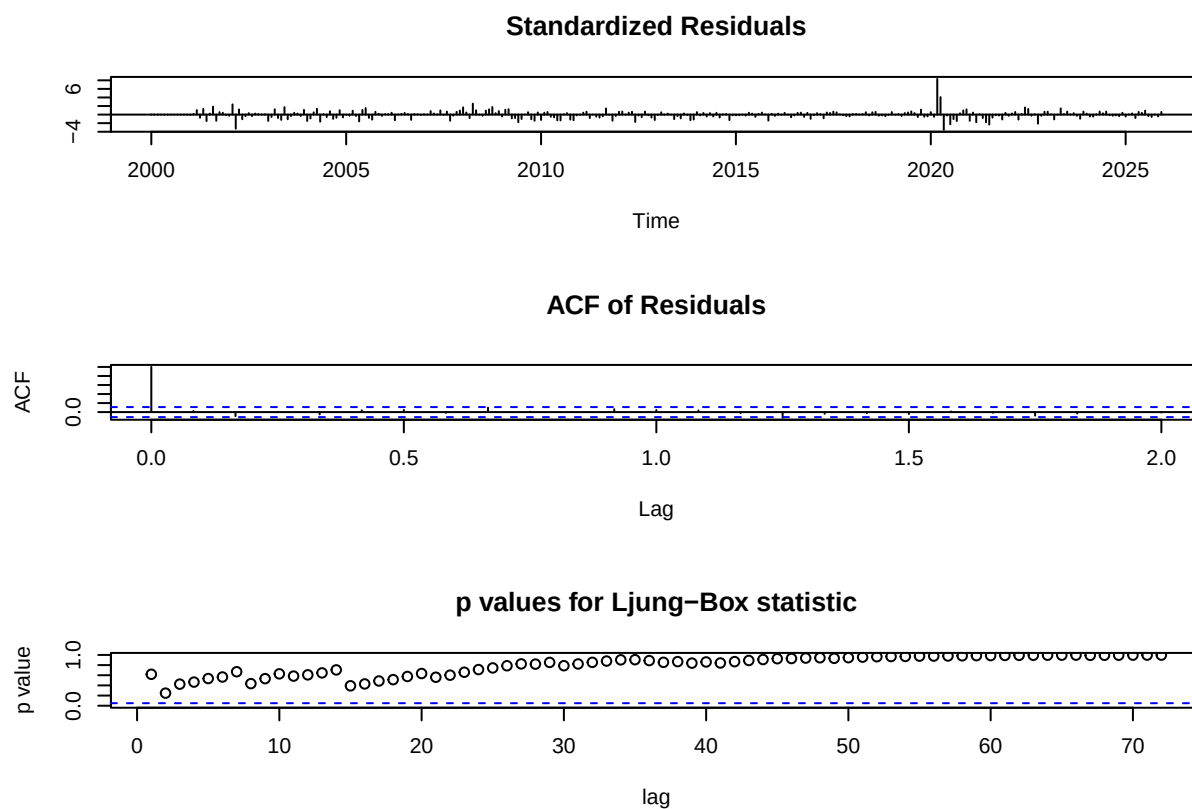
```
par(mfrow = c(1, 2))

# ACF and PACF
acf(res2, main = "ACF", lag.max = 36)
pacf(res2, main = "PACF", lag.max = 36)
```



In both the ACF and PACF lags do not exceed the CIs, hence the residuals appear to be independent.

```
# Ljung-Box test  
tsdiag(mod2, gof.lag = 72)
```



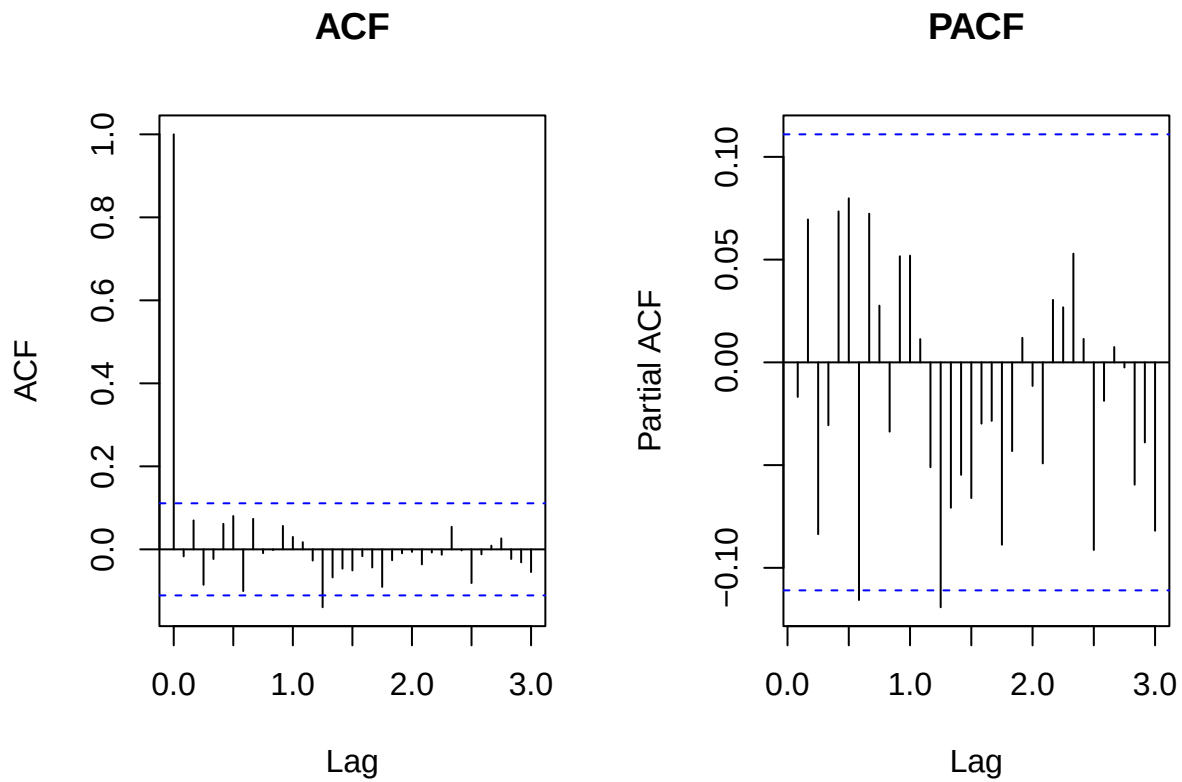
$p - value > 0.05$ for all lags, hence there is no evidence of dependence between the residuals.

Model 3

We plot the ACF and PACF of the residuals of model 3.

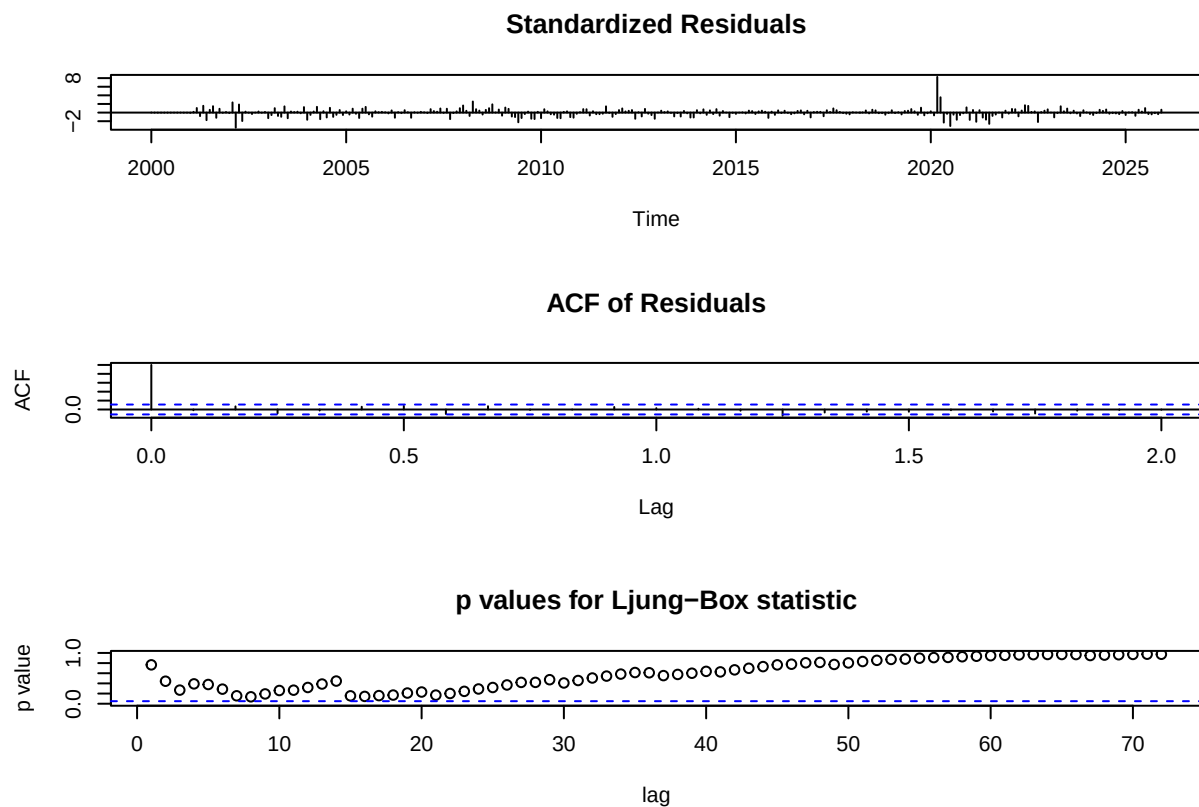
```
par(mfrow = c(1, 2))

# ACF and PACF
acf(res3, main = "ACF", lag.max = 36)
pacf(res3, main = "PACF", lag.max = 36)
```



In both the ACF and PACF few lags barely exceed the CIs. Again, we are considering 95% CI, hence it's acceptable for 1 out 20 to exceed the CIs. Therefore residuals appear to be independent.

```
# Ljung-Box test  
tsdiag(mod3, gof.lag = 72)
```



$p - value > 0.05$ for all lags, hence there is no evidence of dependence between the residuals.

E - Model fit measures

Instructions: compute model fit measures (AIC and BIC).

We will compute the AIC and BIC for the models we kept after the residual analysis, which are $SARIMA(0, 2, 7)(0, 1, 1)_{12}$ and $SARIMA(1, 2, 1)(0, 1, 1)_{12}$.

```
# Creating a table for easier comparison
comparison <- data.frame(
  Model = c("Model 2", "Model 3"),
  AIC = c(AIC(mod2), AIC(mod3)),
  BIC = c(BIC(mod2), BIC(mod3))
)
comparison
```

```
##      Model      AIC      BIC
## 1 Model 2 -1754.863 -1736.377
## 2 Model 3 -1750.277 -1735.488
```

Model 2 has both lower AIC and BIC compared to model 3, indicating better fit to the data.

F - Refitting the model

Instructions: fit the model using all data and then excluding the last 12 observations.

First we define the trade dataset, holding out the last 12 observations.

```
train <- window(lnserie, end = c(2024, 12))
```

Model 2

We fit model 2 on the full dataset and on the train dataset.

```
# Full dataset
mod2 <- arima(lnserie, order = c(0, 2, 7),
              seasonal = list(order = c(0, 1, 1), period = 12),
              fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
mod2

##
## Call:
## arima(x = lnserie, order = c(0, 2, 7), seasonal = list(order = c(0, 1, 1), period = 12),
##       fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
##
## Coefficients:
##          ma1  ma2          ma3  ma4  ma5  ma6          ma7          sma1
##      -0.4731   0  -0.2607   0   0   0  -0.1298  -0.8840
## s.e.   0.0539   0   0.0517   0   0   0   0.0627   0.0418
##
## sigma^2 estimated as 0.0001437:  log likelihood = 882.43,  aic = -1754.86

# Train dataset
mod2_train <- arima(train, order = c(0, 2, 7),
                   seasonal = list(order = c(0, 1, 1), period = 12),
                   fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
mod2_train

##
## Call:
## arima(x = train, order = c(0, 2, 7), seasonal = list(order = c(0, 1, 1), period = 12),
##       fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))
##
## Coefficients:
##          ma1  ma2          ma3  ma4  ma5  ma6          ma7          sma1
##      -0.4695   0  -0.2591   0   0   0  -0.1335  -0.8862
## s.e.   0.0556   0   0.0525   0   0   0   0.0641   0.0428
##
## sigma^2 estimated as 0.0001479:  log likelihood = 842.08,  aic = -1674.17
```

Notice that AIC is lower when fitting the model on the whole time series.

Model 3

We fit model 3 on the full dataset and on the train dataset.

```
mod3 <- arima(lnserie, order = c(1, 2, 1),
              seasonal = list(order = c(0, 1, 1), period = 12))
mod3
```

```
##
## Call:
## arima(x = lnserie, order = c(1, 2, 1), seasonal = list(order = c(0, 1, 1), period = 12))
##
## Coefficients:
##          ar1          ma1          sma1
##      0.4413   -0.8762   -0.8852
## s.e.  0.0924    0.0596    0.0410
##
## sigma^2 estimated as 0.0001474:  log likelihood = 879.14,  aic = -1750.28
```

```
mod3_train <- arima(train, order = c(1, 2, 1),
                   seasonal = list(order = c(0, 1, 1), period = 12))
mod3_train
```

```
##
## Call:
## arima(x = train, order = c(1, 2, 1), seasonal = list(order = c(0, 1, 1), period = 12))
##
## Coefficients:
##          ar1          ma1          sma1
##      0.4422   -0.8748   -0.8878
## s.e.  0.0944    0.0610    0.0421
##
## sigma^2 estimated as 0.0001517:  log likelihood = 838.97,  aic = -1669.93
```

Again, the AIC is lower when fitting the model on the whole time series.

G - Point forecasts and estimated CIs

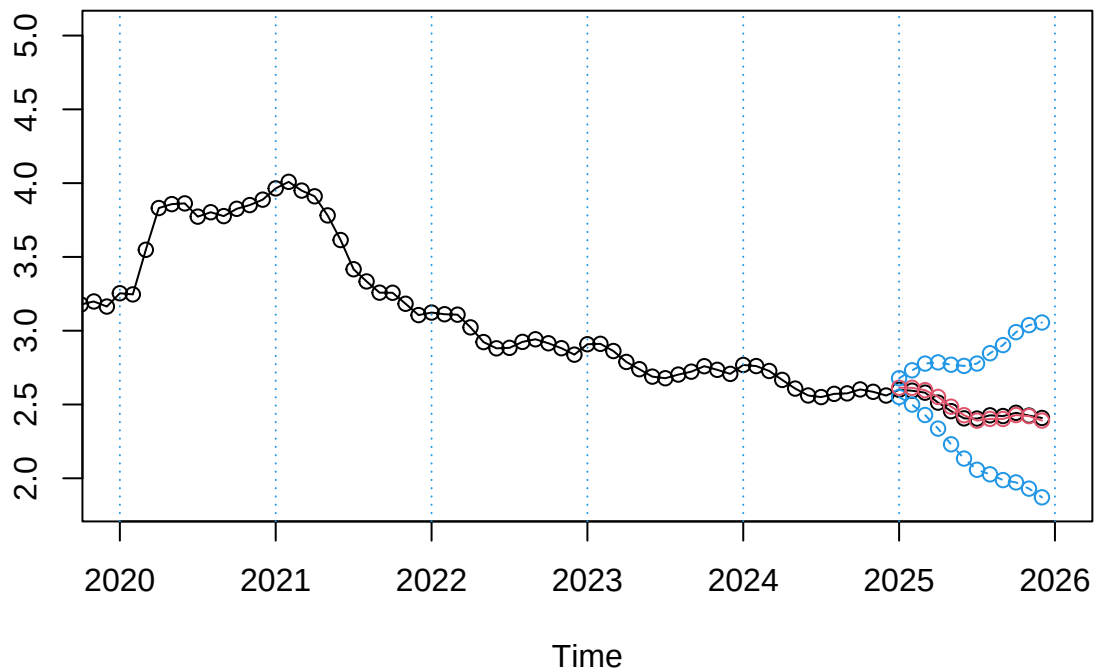
Instructions: for the model fitted without the last 12 observations, obtain point forecasts and the corresponding 95% confidence intervals for the final year. Plot the original series (last 5 years) with the forecasts and confidence intervals superimposed.

Model 2

We obtain out-of-sample forecasts of model 2.

```
# Forecasting model 2
fc2 <- predict(mod2_train, n.ahead = 12)
pr2 <- exp(fc2$pred) # Point forecast
up2 <- exp(fc2$pred + 2 * fc2$se) # Upper 95% CI
low2 <- exp(fc2$pred - 2 * fc2$se) # Lower 95% CI
```

```
ts.plot(serie, low2, up2, pr2, lty = c(1, 2, 2, 1),
       col = c(1, 4, 4, 2), xlim = c(2020, 2026), type = "o")
abline(v = 2020:2026, lty = 3, col = 4)
```

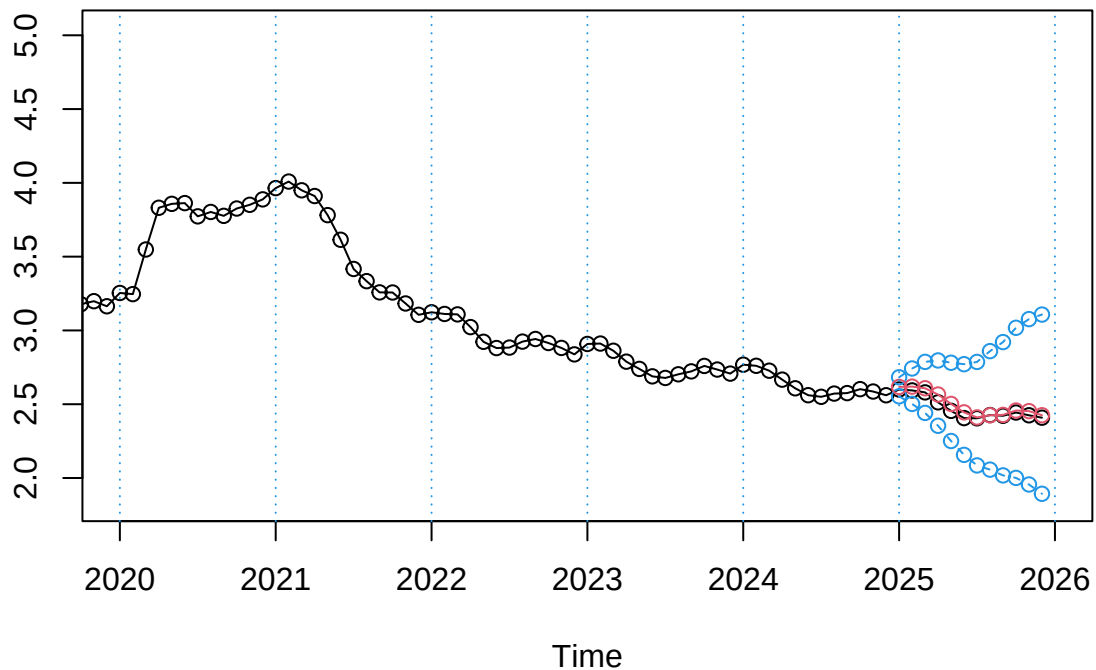


Model 3

We obtain out-of-sample forecasts of model 3.

```
# Forecasting model 2
fc3 <- predict(mod3_train, n.ahead = 12)
pr3 <- exp(fc3$pred) # Point forecast
up3 <- exp(fc3$pred + 2 * fc3$se) # Upper 95% CI
low3 <- exp(fc3$pred - 2 * fc3$se) # Lower 95% CI

ts.plot(serie, low3, up3, pr3, lty = c(1, 2, 2, 1),
       col = c(1, 4, 4, 2), xlim = c(2020, 2026), type = "o")
abline(v = 2020:2026, lty = 3, col = 4)
```

H - Forecasting accuracy measures

Instructions: compute forecasting accuracy measures (RMSE, MAE, RMSPE, and MAPE). These provide insights into the accuracy of the predictions.

We compute the forecasting accuracy measures. These will be reported in a final table in question 8 to compare the two models and select the best one to generate predictions.

Model 2

We compute accuracy measures for model 2.

```
# Pulling the last 12 observations
obs <- tail(serie, 12)

# Calculating the accuracy measures
RMSE2 <- sqrt(mean((obs - pr2) * (obs - pr2)))
MAE2 <- mean(abs(obs - pr2))
RMSPE2 <- sqrt(mean(((obs - pr2) / obs) * (obs - pr2) / obs)) * 100
MAPE2 <- mean(abs(obs - pr2) / obs) * 100
```

Model 3

We compute accuracy measures for model 3.

```
# Pulling the last 12 observations
obs <- tail(serie, 12)

# Calculating the accuracy measures
RMSE3 <- sqrt(mean((obs - pr3) * (obs - pr3)))
MAE3 <- mean(abs(obs - pr3))
RMSPE3 <- sqrt(mean(((obs - pr3) / obs)*(obs - pr3) / obs)) * 100
MAPE3 <- mean(abs(obs - pr3) / obs) * 100
```

I - Average width of CIs

Instructions: calculate the average width of the prediction confidence intervals. This is a measure of forecast precision.

We compute the average CI width to measure precision. These will be reported in a final table in question 8 to compare the two models and select the best one to generate predictions.

Model 2

We compute average CI width for model 2.

```
meanCI2 <- mean(up2 - low2)
```

Model 3

We compute average CI width for model 3.

```
meanCI3 <- mean(up3 - low3)
```

8 - Model selection and forecast

Instructions: based on all previous information, which of the two proposed models would you select as the best? For the selected model fitted to the entire dataset, generate forecasts and confidence intervals for the next year.

Comparing accuracy

First, we will compare the accuracy measures obtained from model 2 and model 3 by building a comparison table.

```
# Building the comparison table
accuracy_table <- data.frame(
  Metric = c("RMSE", "MAE", "RMSPE (%)", "MAPE (%)"),
  Model2 = c(RMSE2, MAE2, RMSPE2, MAPE2),
  Model3 = c(RMSE3, MAE3, RMSPE3, MAPE3)
)
accuracy_table
```

```
##      Metric      Model2      Model3
## 1      RMSE 0.02134319 0.02832914
## 2      MAE 0.01963657 0.02351474
## 3 RMSPE (%) 0.86224226 1.14138459
## 4 MAPE (%) 0.79378193 0.94698485
```

Errors of model 2 are smaller compared to errors of model 3, hence model 2 is the more accurate model.

Comparing precision

Now, we will compare the average CI width obtained from model 2 and model 3.

```
precision_table <- data.frame(
  Metric = c("Mean CI Width"),
  Model2 = c(meanCI2),
  Model3 = c(meanCI3)
)
precision_table
```

```
##      Metric      Model2      Model3
## 1 Mean CI Width 0.674184 0.6718534
```

Model 3 has slightly tighter CIs, achieving better precision compared to model 2.

Model decision and prediction

To recap, model 2 is more accurate, but model 3 is more precise. Hence, if we base our decision on accuracy, we should select model 2, while if we base our decision on precision we should select model 3. We will base our decision on accuracy, hence we will select model 2 which is $SARIMA(0, 2, 7)(0, 1, 1)_{12}$ without the inclusion of the following coefficients:

- ma2
- ma4
- ma5
- ma6

Now, we retrain the model on the whole dataset and generate point forecasts and confidence intervals for year 2026.

```
# Retraining on the whole dataset
mod2 <- arima(lnserie, order = c(0, 2, 7),
              seasonal = list(order = c(0, 1, 1), period = 12),
              fixed = c(NA, 0, NA, 0, 0, 0, NA, NA))

# Forecasting
fc <- predict(mod2, n.ahead = 12)
pr <- exp(fc$pred) # Point forecast
up <- exp(fc$pred + 2 * fc$se) # Upper 95% CI
low <- exp(fc$pred - 2 * fc$se) # Lower 95% CI

# Plotting
```

```
ts.plot(serie, low, up, pr, lty = c(1, 2, 2, 1),
       col = c(1, 4, 4, 2), xlim = c(2020, 2027), type = "o")
abline(v = 2020:2027, lty = 3, col = 4)
```

